

Read this table as a work plan. Every row implies a control the platform must provide, or each team will invent its own and none will agree.

What does an LLM platform actually look like?

Most LLM platforms converge on the same seven layers, whether the team planned it or arrived there by accretion. Naming them helps, because it tells you where a given control belongs.

The gateway earns its place first, because only four controls can sit together there:

- Identity and tenancy, so every request carries an owner.
- Token metering, so cost is attributable before the invoice arrives.
- Guardrails on input and output, applied once instead of per service.
- Routing and fallback, so you can switch providers without touching application code.

Teams that skip it end up with no cost attribution, no kill switch, and no failover.

What happens inside a single request?

The cost and latency levers all sit somewhere on the request

path, so it is worth tracing once end to end.

Three facts from that path explain most of what you will later see on dashboards:

- Prefill processes the whole prompt at once, is compute bound, and sets time to first token. Long context and slow first tokens travel together.
- Decode generates one token at a time, is bound by memory bandwidth, and sets the inter-token delay users experience as typing speed.
- The key value cache holding attention state for every active request grows with context length and consumes GPU memory.

That third point is the one to keep. Concurrency on an inference server is limited by memory, not by CPU, so when the cache fills, tail latency degrades sharply rather than gradually.

What has to be versioned and deployed?

A conventional service has code and config. An LLM service has at least eight things that change behaviour in production, and a change to any one of them is a deployment:

- Model weights and version, plus the quantization and engine config used to serve them.

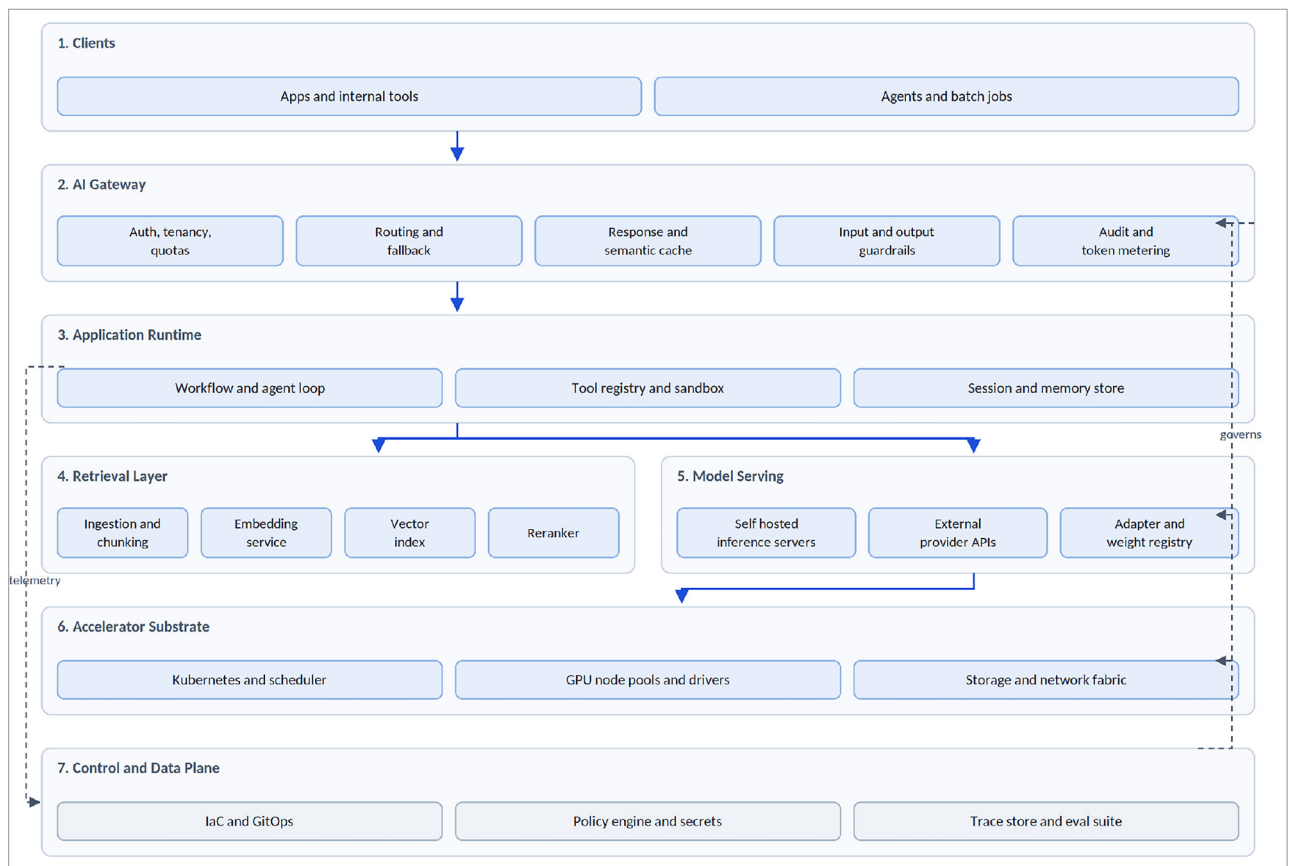


Figure 1: Reference architecture. The gateway and the control plane are the two layers platform teams most often skip, and the two they most often have to retrofit under pressure